University Management System

ER Diagram Documentation

1NF $\rightarrow$ 3NF Normalization Process

Note: Visual ER diagrams are generated using MySQL Workbench's Reverse Engineering feature from schema.sql. This document describes the normalization process, entity descriptions, and relationship specifications.

1. First Normal Form (1NF) — Unnormalized Design

The initial design was modeled as a single flat table (_demo_1nf_university_record) containing all university data in one row per student-course-book combination. This table is included in schema.sql for demonstration purposes.

1.1 Problems in 1NF

Problem	Description
Redundancy	Student personal info (name, email, address, program) is repeated for every course they take. Instructor info repeats for every section.
Update Anomaly	Changing a student's address requires updating every row for that student.
Insertion Anomaly	Cannot add a new department without associating it with a student record.
Deletion Anomaly	Deleting the last student in a course may lose the course information entirely.
No Referential Integrity	No foreign keys — any value can be placed in any column.

1.2 1NF Table Structure

The _demo_1nf_university_record table has 34 columns including: record_id, student_id, student_first_name, student_last_name, student_email, student_phone, student_dob, student_gender, student_address, student_cgpa, program_name, degree_type, department_name, department_phone, course_code, course_name, course_credits, instructor_name, instructor_email, semester, section_year, schedule_days, schedule_time, room, grade, grade_points, fee_total, fee_paid, fee_status, hostel_name, hostel_room_number, book_title, book_isbn, issue_date, return_date.

2. Normalization Process: 1NF → 2NF → 3NF

2.1 Step 1: 1NF → 2NF (Eliminate Partial Dependencies)

Partial dependencies exist where non-key attributes depend on only part of a composite key. We decompose the flat table into separate entities based on functional dependencies:

Data Group	Functional Dependency	Result
Student info	student_id → first_name, last_name, email, dob, gender, address, cgpa	Extracted into: student table
Department info	department_name → department_phone, description, office	Extracted into: department table with surrogate department_id
Program info	program_name + degree_type → department, duration, credits	Extracted into: program table with FK to department
Course info	course_code → name, description, credits, department	Extracted into: course table with FK to department
Section info	section_id → course, instructor, semester, year, schedule, room	Extracted into: section table with FKs to course and instructor
Fee info	student_id + semester + year → total, paid, status	Extracted into: fee table with FK to student
Book info	isbn → title, author, publisher, copies	Extracted into: book table
Hostel info	hostel_name → type, rooms, warden	Extracted into: hostel table

2.2 Step 2: 2NF → 3NF (Eliminate Transitive Dependencies)

Transitive dependencies exist where a non-key attribute depends on another non-key attribute. These are resolved by further decomposition:

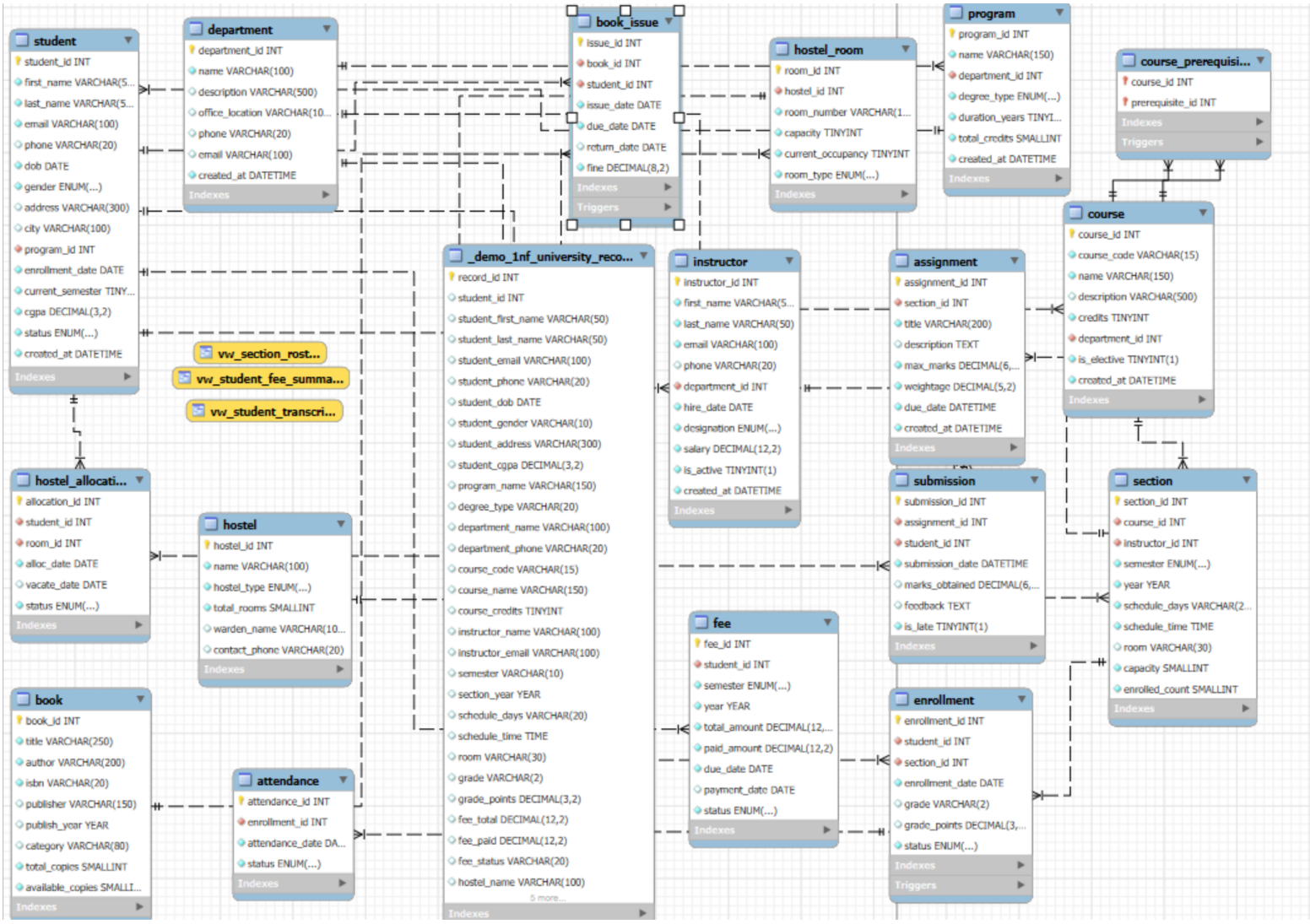
Transitive Dependency	Resolution
student.program_name → department_name	program is extracted as a separate entity; student references program_id (FK), and program references department_id (FK). No transitive path.
section.instructor_name → instructor_email, department	instructor is a separate entity; section references instructor_id (FK).
enrollment.grade → section → course	enrollment references section_id; section references course_id. Grade depends only on the enrollment key.
hostel_room.hostel_name → hostel_type, warden	hostel_room references hostel_id (FK); hostel details are in their own table.
book_issue.book_title → author, isbn	book_issue references book_id (FK); book details are in their own table.

3. Final 3NF Entity Descriptions

Entity (Table)	Description	Size
department	Academic departments in the university.	6 columns
program	Degree programs offered within departments.	7 columns
instructor	Faculty members with department affiliation.	11 columns
student	Enrolled students with program and status tracking.	15 columns
course	Courses offered across departments.	8 columns
course_prerequisite	M:N self-referencing junction for prerequisites.	2 columns (composite PK)
section	Course offerings in specific semesters.	10 columns
enrollment	Student-section registrations with grade.	7 columns
attendance	Daily attendance per enrollment.	4 columns
assignment	Graded work items per section.	8 columns
submission	Student submissions for assignments.	7 columns
fee	Semester-wise fee billing and payment tracking.	9 columns
book	Library catalog entries.	9 columns
book_issue	Book lending transactions with fine tracking.	7 columns
hostel	Residential hostel buildings.	6 columns
hostel_room	Individual rooms within hostels.	6 columns
hostel_allocation	Student-to-room assignments.	6 columns

4. Relationships & Cardinalities

All relationships use Crow's Foot notation when visualized in MySQL Workbench. The following table documents every relationship in the schema:

Relationship	Cardinality	Description
department → program	1:N	A department offers many programs; a program belongs to one department.
department → instructor	1:N	A department has many instructors; an instructor belongs to one department.
department → course	1:N	A department offers many courses; a course belongs to one department.
program → student	1:N	A program has many students; a student is in one program.
course → section	1:N	A course can have multiple sections per semester.
instructor → section	1:N	An instructor teaches many sections; a section has one instructor.
student ↔ section (enrollment)	M:N	Many students enroll in many sections — resolved via enrollment table.
enrollment → attendance	1:N	An enrollment has many attendance records.
section → assignment	1:N	A section has many assignments.
student ↔ assignment (submission)	M:N	Many students submit many assignments — resolved via submission table.
student → fee	1:N	A student has many fee records (one per semester).
course ↔ course (prerequisite)	M:N self-ref	A course can require many prerequisites and be a prerequisite for many.
student ↔ book (book_issue)	M:N	Many students borrow many books — resolved via book_issue table.
hostel → hostel_room	1:N	A hostel has many rooms; a room belongs to one hostel.
student ↔ hostel_room (allocation)	M:N	Many students in many rooms over time — resolved via hostel_allocation.

5. Generating Visual ER Diagrams in MySQL Workbench